

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND
MICROELECTRONICS

DEPARTMENT OF SOFTWARE ENGINEERING AND
AUTOMATIC

ANDREI ELCISCEV, FAF-243

Laboratory work nr. 1 on Cryptography and Security

Caesar Cipher and Caesar Cipher with Permutation over the
Romanian Alphabet

Checked by:
Zaica Maia

Chisinau 2026

Contents

1	Objectives	2
2	Short Theory	2
2.1	The Caesar Cipher	2
2.2	Romanian Alphabet Encoding	3
2.3	Caesar Cipher with a Permutation	3
3	Implementation	3
3.1	Romanian Alphabet	3
3.2	Normalization and Validation	4
3.3	Numeric Key Validation	4
3.4	Encryption and Decryption	5
3.5	Building the Permuted Alphabet	5
3.6	Keyword Validation	6
3.7	Permuted-Alphabet Encryption	7
4	Results	7
4.1	Task 1.1 – Simple Caesar Cipher	7
4.2	Task 1.2 – Caesar Cipher with Keyword Permutation	9
5	Conclusions	9
6	Source Code	10

1 Objectives

The purpose of this laboratory work is to understand how the Caesar cipher works, why its small key space makes it insecure, how a keyword can be used to permute the alphabet, and how both versions can be implemented for the Romanian alphabet.

The objectives are:

- to implement the Caesar cipher for the Romanian alphabet;
- to understand encryption and decryption using modular arithmetic;
- to demonstrate the weakness of the small Caesar key space;
- to implement a keyword-based permutation of the Romanian alphabet;
- to validate numeric keys, keywords, and message characters;
- to test encryption and decryption for both algorithms.

2 Short Theory

2.1 The Caesar Cipher

The Caesar cipher is a substitution cipher in which every plaintext letter is replaced by the letter found a fixed number of positions further along the alphabet. The numeric secret key k is the shift used for both encryption and decryption.

For an alphabet of length n :

$$c = e_k(x) = (x + k) \bmod n$$

$$m = d_k(y) = (y - k) \bmod n.$$

Here x and y are the numeric positions of plaintext and ciphertext letters, respectively. For this laboratory, the Romanian alphabet has $n = 31$ letters, so:

$$k \in \{1, 2, \dots, 30\}.$$

The value $k = 0$ is excluded because it leaves the message unchanged. During decryption, if the result of $y - k$ is negative, modulo arithmetic brings it back into the range $0, \dots, n - 1$.

The main weakness of the Caesar cipher is its small key space. With the Romanian alphabet there are only 30 possible non-zero keys, so an attacker can simply test all keys in a brute-force attack.

2.2 Romanian Alphabet Encoding

The assignment requires the following explicit alphabet and codes:

A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z

with codes from 0 to 30. The implementation stores these letters explicitly and therefore does not use ASCII or Unicode code points to calculate the shifts.

The program also accepts the cedilla variants Ș/ș and Ț/ț and normalizes them to Ș/ș and Ț/ț before processing.

2.3 Caesar Cipher with a Permutation

The second version first creates a permutation of the alphabet using a keyword. The distinct letters of the keyword are placed first, preserving their order. All letters that have not appeared in the keyword are then appended in their normal Romanian-alphabet order.

The Caesar shift is applied to this new alphabet:

$$c = (x + k) \bmod n$$

and decryption is:

$$m = (y - k) \bmod n.$$

The keyword must contain only Romanian alphabet letters and must contain at least seven characters after normalization.

A keyword permutation makes exhaustive search substantially harder because there are many possible alphabet arrangements. For a 31-letter alphabet, the theoretical number of permutations is $31!$, combined with 30 possible shifts. However, the resulting system is still a monoalphabetic substitution cipher. Each plaintext letter is always mapped to the same ciphertext letter, so frequency analysis remains possible.

3 Implementation

The application was implemented in C# as a console program. The implementation is divided into an alphabet utility, a Caesar cipher class, and a console user-interface class.

3.1 Romanian Alphabet

The required alphabet is explicitly defined:

```

public static readonly char[] RomanianAlphabet = new char[]
{
    'A', '\u0102', '\u00C2',
    'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I',
    '\u00CE',
    'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R',
    'S', '\u0218', 'T', '\u021A',
    'U', 'V', 'W', 'X', 'Y', 'Z'
};

```

This array provides the exact positions 0–30 required by the assignment.

3.2 Normalization and Validation

Before encryption, the program converts text to uppercase, removes whitespace, and normalizes alternative cedilla forms:

```

s = s.Replace('\u015E', '\u0218')
    .Replace('\u015F', '\u0219')
    .Replace('\u0162', '\u021A')
    .Replace('\u0163', '\u021B');

s = s.ToUpperInvariant();

return new string(
    s.Where(c => !char.IsWhiteSpace(c)).ToArray());

```

Each resulting character is then checked against the explicitly defined alphabet:

```

foreach (var c in s)
{
    if (!RomanianAlphabet.Contains(c))
    {
        invalid = c;
        return false;
    }
}

```

If validation fails, the program reports the rejected character and displays the allowed letters.

3.3 Numeric Key Validation

The first key must be an integer from 1 to 30:

```

if (int.TryParse(s, out int k) &&
    k >= 1 && k <= Alphabet.RomanianAlphabet.Length - 1)
{
    return k;
}

Console.WriteLine(
    "Invalid key. Allowed values are integers from 1 to 30.");

```

This prevents invalid numeric values from being used by the cipher.

3.4 Encryption and Decryption

The simple Caesar cipher calls the same transformation method with opposite shift directions:

```

public static string EncryptSimple(string plain, int k)
{
    return TransformByAlphabet(
        plain, k, Alphabet.RomanianAlphabet);
}

public static string DecryptSimple(string cipher, int k)
{
    return TransformByAlphabet(
        cipher, -k, Alphabet.RomanianAlphabet);
}

```

The actual shift is performed using the position of each letter in the selected alphabet:

```

int shifted = (pos + shift) % n;

if (shifted < 0)
    shifted += n;

outChars[i] = alphabet[shifted];

```

The explicit correction for negative values is required during decryption.

3.5 Building the Permuted Alphabet

The keyword permutation is constructed using a `HashSet<char>` to ensure that every keyword letter is included only once:

```

var seen = new HashSet<char>();
var list = new List<char>();

foreach (var ch in keyword)
{
    if (!seen.Contains(ch) &&
        Alphabet.RomanianAlphabet.Contains(ch))
    {
        seen.Add(ch);
        list.Add(ch);
    }
}

foreach (var ch in Alphabet.RomanianAlphabet)
{
    if (!seen.Contains(ch))
        list.Add(ch);
}

return list.ToArray();

```

The result contains all 31 Romanian letters exactly once. The application displays the resulting permutation so it can be checked manually.

3.6 Keyword Validation

The second key is normalized and then checked for length and valid characters:

```

kw = Alphabet.NormalizeAndStrip(kw);

if (kw.Length < 7)
{
    Console.WriteLine(
        "Keyword too short. It must have at least 7 letters.");
    continue;
}

if (!Alphabet.ValidateTextChars(kw, out char invalid))
{
    Console.WriteLine(
        $"Invalid character '{invalid}' in keyword.");
    continue;
}

```

```
}
```

This ensures that only valid Romanian keywords satisfying the minimum length are accepted.

3.7 Permuted-Alphabet Encryption

The second algorithm reuses the same transformation method with the generated permutation:

```
public static string EncryptWithPermuted(
    string plain, int k, char[] perm)
{
    return TransformByAlphabet(plain, k, perm);
}

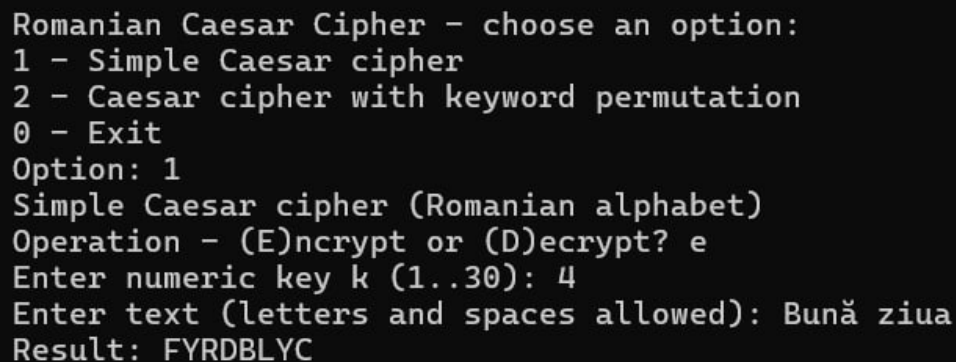
public static string DecryptWithPermuted(
    string cipher, int k, char[] perm)
{
    return TransformByAlphabet(cipher, -k, perm);
}
```

This design avoids duplicating the transformation logic.

4 Results

The program was tested with valid encryption and decryption operations for both tasks, as well as the required invalid-input cases. The following spaces are reserved for screenshots of the running application.

4.1 Task 1.1 – Simple Caesar Cipher



```
Romanian Caesar Cipher – choose an option:
1 – Simple Caesar cipher
2 – Caesar cipher with keyword permutation
0 – Exit
Option: 1
Simple Caesar cipher (Romanian alphabet)
Operation – (E)ncrypt or (D)ecrypt? e
Enter numeric key k (1..30): 4
Enter text (letters and spaces allowed): Bună ziua
Result: FYRDBLYC
```

Figure 1: Simple Caesar cipher encryption

```

Romanian Caesar Cipher - choose an option:
1 - Simple Caesar cipher
2 - Caesar cipher with keyword permutation
0 - Exit
Option: 1
Simple Caesar cipher (Romanian alphabet)
Operation - (E)ncrypt or (D)ecrypt? d
Enter numeric key k (1..30): 4
Enter text (letters and spaces allowed): FYRDBLYC
Result: BUNĂZIUA

```

Figure 2: Simple Caesar cipher decryption

```

Romanian Caesar Cipher - choose an option:
1 - Simple Caesar cipher
2 - Caesar cipher with keyword permutation
0 - Exit
Option: 1
Simple Caesar cipher (Romanian alphabet)
Operation - (E)ncrypt or (D)ecrypt? e
Enter numeric key k (1..30): 42
Invalid key. Allowed values are integers from 1 to 30.
Enter numeric key k (1..30): |

```

Figure 3: Invalid numeric key validation

```

Romanian Caesar Cipher - choose an option:
1 - Simple Caesar cipher
2 - Caesar cipher with keyword permutation
0 - Exit
Option: 1
Simple Caesar cipher (Romanian alphabet)
Operation - (E)ncrypt or (D)ecrypt? 1
Please enter E for encryption or D for decryption.
Operation - (E)ncrypt or (D)ecrypt? e
Enter numeric key k (1..30): 4
Enter text (letters and spaces allowed): %ABRACADABRA^
Invalid character '%' (U+0025). Allowed letters are: A Ă Â B C D

```

Figure 4: Invalid message character validation

4.2 Task 1.2 – Caesar Cipher with Keyword Permutation

```
Romanian Caesar Cipher - choose an option:
1 - Simple Caesar cipher
2 - Caesar cipher with keyword permutation
0 - Exit
Option: 2
Caesar cipher with keyword permutation (Romanian alphabet)
Operation - (E)ncrypt or (D)ecrypt? e
Enter numeric key k (1..30): 4
Enter keyword (Romanian letters only, at least 7 characters): Elcişcev
Permuted alphabet:
ELCIŞVAĂÂBDFGHÎJKMNOPQRSTȚUWXYZ
Enter text (letters and spaces allowed): Bună seara
Result: HZRFWŞDUD
```

Figure 5: Keyword-permutation Caesar cipher encryption

```
Romanian Caesar Cipher - choose an option:
1 - Simple Caesar cipher
2 - Caesar cipher with keyword permutation
0 - Exit
Option: 2
Caesar cipher with keyword permutation (Romanian alphabet)
Operation - (E)ncrypt or (D)ecrypt? d
Enter numeric key k (1..30): 4
Enter keyword (Romanian letters only, at least 7 characters): elcişcev
Permuted alphabet:
ELCIŞVAĂÂBDFGHÎJKMNOPQRSTȚUWXYZ
Enter text (letters and spaces allowed): HZRFWŞDUD
Result: BUNĂSEARA
```

Figure 6: Keyword-permutation Caesar cipher decryption and displayed alphabet

```
Romanian Caesar Cipher - choose an option:
1 - Simple Caesar cipher
2 - Caesar cipher with keyword permutation
0 - Exit
Option: 2
Caesar cipher with keyword permutation (Romanian alphabet)
Operation - (E)ncrypt or (D)ecrypt? e
Enter numeric key k (1..30): 4
Enter keyword (Romanian letters only, at least 7 characters): andrei
Keyword too short. It must have at least 7 letters after removing spaces.
Enter keyword (Romanian letters only, at least 7 characters): |
```

Figure 7: Invalid keyword length validation

These tests demonstrate that the implementation performs the required encryption and decryption operations and rejects invalid input according to the assignment requirements.

5 Conclusions

The laboratory work provided practical experience with classical substitution ciphers and demonstrated the relationship between an algorithm's key space and its resistance to

brute-force attacks. The simple Caesar cipher is easy to implement and understand, but its 30 possible non-zero keys for the Romanian alphabet make it unsuitable for secure communication.

The keyword-permutation version increases the number of possible configurations by changing the alphabet order before applying the Caesar shift. This makes exhaustive search considerably more difficult than for the ordinary Caesar cipher. Nevertheless, it remains a monoalphabetic substitution cipher, so frequency analysis can still be used to attack it.

The implementation also demonstrated the importance of using an explicit alphabet representation when an assignment specifies a custom character order. Input normalization and validation were implemented so that spaces, Romanian diacritics, invalid characters, invalid numeric keys, and short keywords are handled correctly.

Overall, the laboratory helped reinforce concepts such as modular arithmetic, substitution ciphers, key spaces, brute-force attacks, alphabet permutations, frequency analysis, and input validation.

6 Source Code

The complete source code is available in the GitHub repository:

GitHub repository: <https://github.com/EXakiToR/cs>

The repository contains the Romanian alphabet implementation, normalization and validation functions, the simple Caesar cipher, the keyword-permutation Caesar cipher, and the console interface.